

CM2307 Deferral Coursework

Key Items:

Student Number (mandatory):	C24062219	Note: Do not write your name anywhere in the document.
Link to Git (git.cardiff.ac.uk) Repo (mandatory):	https://github.com/Marcus-Tipler/java-oo-principles-and-search-algorithms	Note: Ensure all necessary files are made available and committed to the repository and relevant people have been provided access.
Access to Git Repo has been provided to the teaching and marking team? (mandatory):	Yes (+ public repository) All emails on proforma have been invited to the GITHUB repository as CONTRIBUTORS (more details in ASSUMPTIONS FILE attached).	Note: If the access is not provided, zero (0) marks will be awarded for the component.
Link to Panopto Recording (mandatory):	No mention of video being required, assumptions added to assumption file attached. N / A	Note: Ensure that the demo is clear and showcases the specific functional requirements chosen by you for the application. Ensure relevant people have been provided access to view the recording.
Access to Panopto Recording has been given to the teaching and marking team? (mandatory):	N / A	Note: If the access is not provided, zero (0) marks will be awarded for the component.

I declare that I have used Generative AI software to support me in producing parts of this work. I have used Gen AI in line with the usage defined by the assessment guidance and have maintained academic integrity throughout my work. I have rewritten any Gen AI content in my own words and validated it through using reliable independent sources.

I understand that declaring the use of GenAI tools will not lead to a lower mark, provided that my use is in accordance with the assessment guidelines and I maintain academic integrity.

**All AI chats and general usage have been declared in a folder called “inf. AI Chats for Declaration” in the provided GitHub repository.
You may request further Information if required.**

It is my responsibility to retain copies of all GenAI prompts and outputs after submission in case of any concerns over academic misconduct, as I may be asked to produce these as evidence of how I've used Gen AI as part of any investigation into academic misconduct.

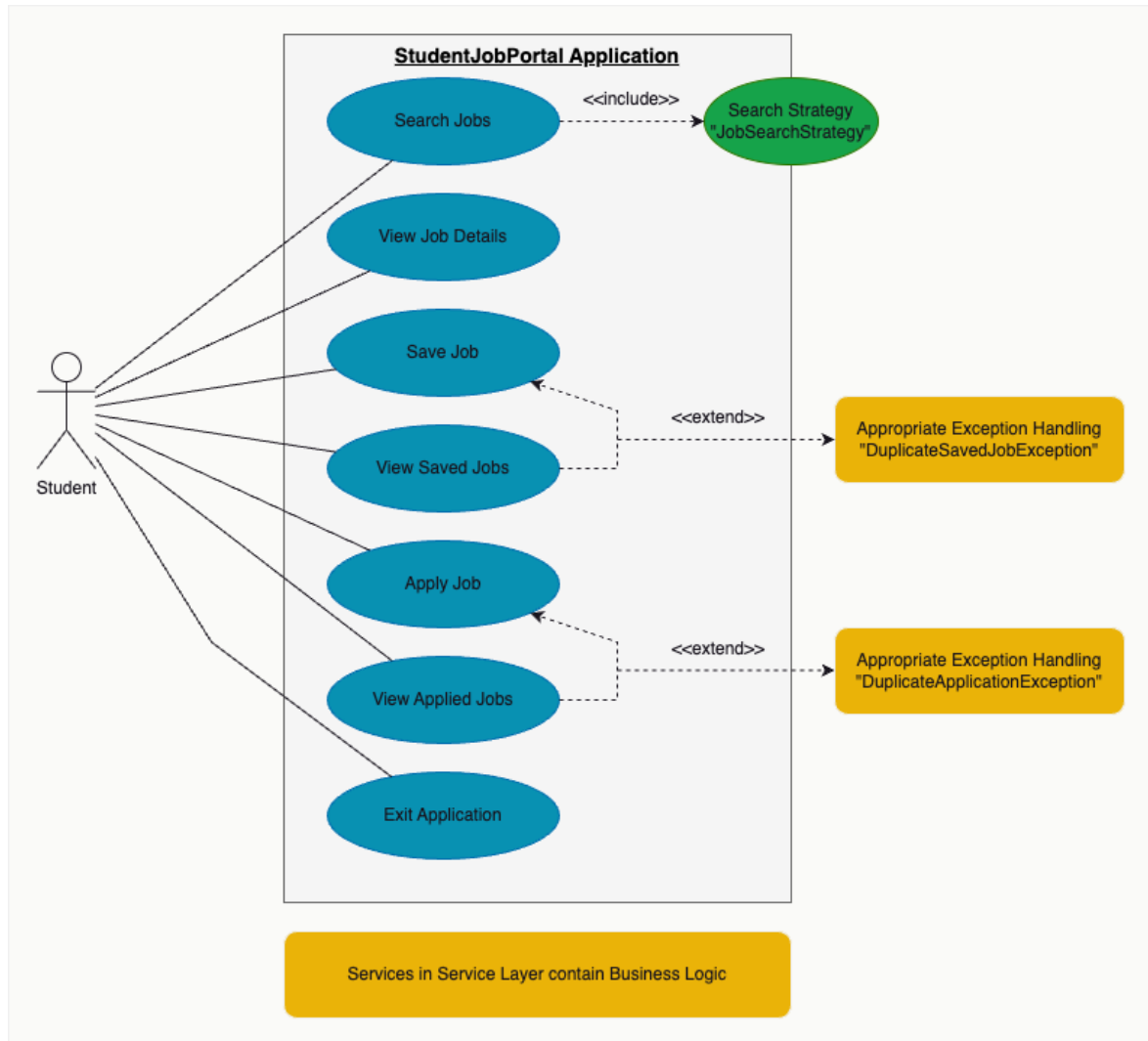
I have also tracked all of my progress and work throughout, using Git Trees.

ASSUMPTIONS FILE

1. I assume that the Test Files & Sample Data (used purely for demonstration purposes) can be generated with AI (all chats provided) due to the fact that I am using Artificial Intelligence as a tool to **save time**. I would not have otherwise wasted time making them and it is not affecting my learning of the subject in any meaningful way.
2. I have taken the liberty of using some **home-made tools** to help me **optimise** this assessment **before publishing it**. This is after the recommendation from my Sr. Personal Tutor DR. LANGBEIN. **Please ignore any remanences of code used for this optimisation process.**
3. I assume, although git.cardiff.ac.uk is suggested, that GitHub is perfectly permissible and that the **use of GitHub over GitLab won't affect my grades**, as access has still been provided as requested to the specified emails in the requirements folder. *"If the code is not provided in git.cardiff.ac.uk as mentioned above **and/or proper access is not provided** to the marking team, it will not be marked and the component associated with the code will be awarded ZERO (0)."*
4. I assume, that **if a corruption on Learning Central were to occur**, and that there is **sufficient evidence that this is NOT my fault**, that a **re-upload of my submission be requested at no cost to my grade** (this happens more often than you expect). I also assume that **meta-data** and **Git Repo History** may be used to **prove I haven't touched the files since the first submission date**.
5. I assume, that although I AM using the provided template, I am allowed to edit it according to my tastes, and **make it look like my other submissions without it costing 10% of my grade**.
6. I have seen **no mention of a Panopto Recording** being required in the provided **proforma**, so I'm assuming no video is required. Even though a **video was required for the previous coursework** and it is mentioned in the above **template**.
7. In GitHub, there are **multiple branches** for this project, I assume **I'll be marked on the one linked above on this template. Not the otherwise MAIN branch** that is also located in the GitHub repository. *(I expect to be graded on the default; REVERTING-CHANGES branch linked).*

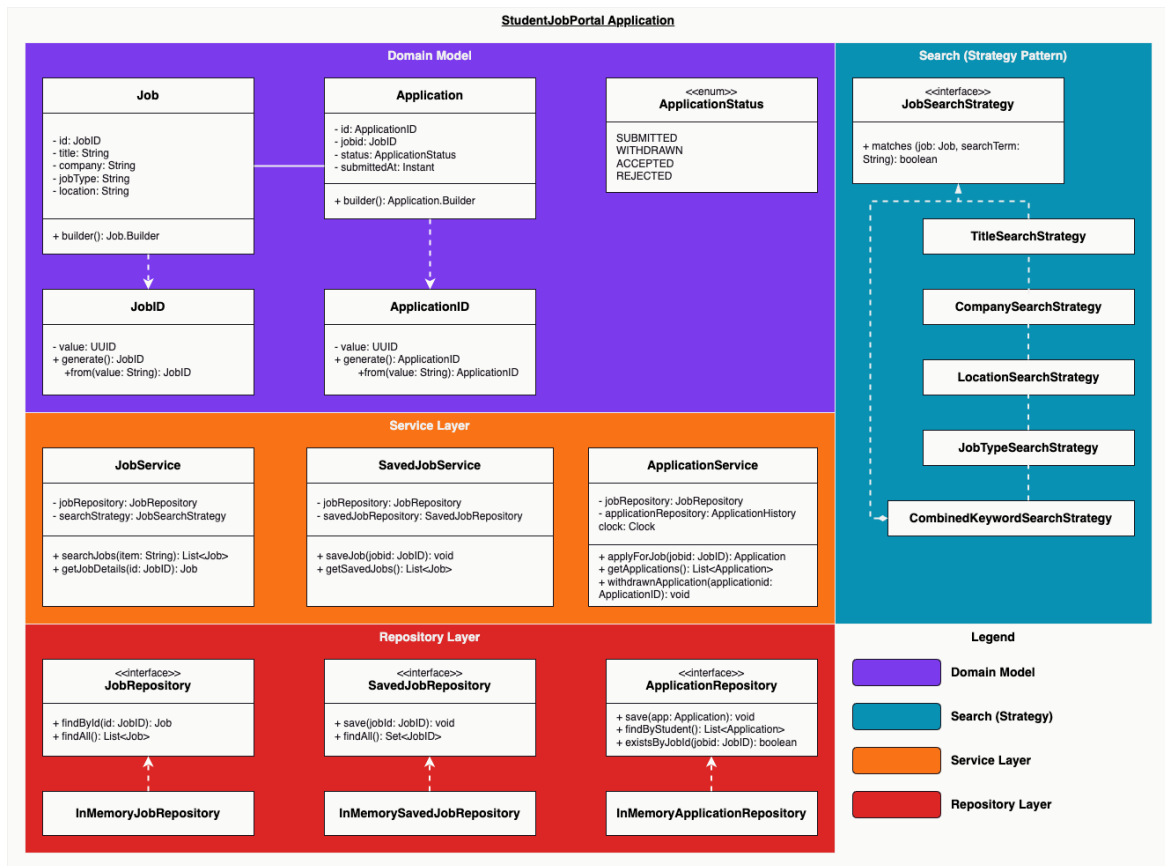
System Design

Some exports of “Use Case”, “Class” & “Sequence” diagrams from the improved implementation of the StudentJobPortal.

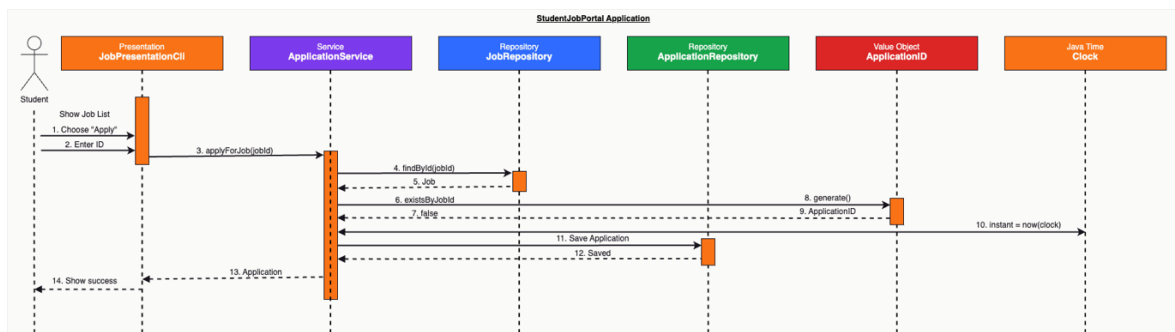


Export of “Use Case Diagram” from Draw.io MacOS taken the 12th Aug 2026.

next page >>



Export of "Class Diagram" from Draw.io MacOS taken the 12th Aug 2026.



Export of "Sequence Diagram" from Draw.io MacOS taken the 12th Aug 2026.

next page >>

Evidence and Evaluation

Part 1:

The improved implementation of the student job portal manages to use object oriented (O.O.) principles to render the program **more maintainable, extendable** and **testable**. It does this thanks to **encapsulation, abstraction** and **polymorphism** supported by several design patterns like **immutability, value objects (UUIDs), composition, dependency injection** and **many more**.

In the original application (*seen bellow in this instance*), the **entire system** was comprised in **one file: “StudentJobPortal.java”** inside its **main()** method. Generally, **having more than 1 responsibility** inside a file makes things **complicated, maintenance heavy, hard to test** and **very difficult to improve** upon in future iterations.

```
while (running) {
    System.out.println(x: "\n1. View Jobs");
    System.out.println(x: "2. Search Jobs");
    System.out.println(x: "3. Save Job");
    System.out.println(x: "4. Apply for Job");
    System.out.println(x: "5. Exit");
    System.out.print(s: "Select option: ");
    int choice = scanner.nextInt();
    scanner.nextLine();

    if (choice == 1) {
        for (int i = 0; i < jobTitles.size(); i++) {
            System.out.println((i + 1) + ". " + jobTitles.get(i) + " | " +
                companies.get(i) + " | " + jobTypes.get(i) + " | " + locations.get(i));
        }
    } else if (choice == 2) {
        System.out.print(s: "Enter keyword: ");
        String keyword = scanner.nextLine().toLowerCase();
        for (int i = 0; i < jobTitles.size(); i++) {
            if (jobTitles.get(i).toLowerCase().contains(keyword) ||
                companies.get(i).toLowerCase().contains(keyword) ||
                locations.get(i).toLowerCase().contains(keyword)) {
                System.out.println("Match: " + jobTitles.get(i));
            }
        }
    } else if (choice == 3) {
        System.out.print(s: "Enter job number to save: ");
        int index = scanner.nextInt() - 1;
        scanner.nextLine();
        if (index >= 0 && index < savedJobs.size()) {
            savedJobs.set(index, element: true);
            System.out.println(x: "Job saved.");
        }
    } else if (choice == 4) {
        System.out.print(s: "Enter job number to apply for: ");
        int index = scanner.nextInt() - 1;
        scanner.nextLine();
        if (index >= 0 && index < jobTitles.size()) {
            System.out.println("Application submitted for " + jobTitles.get(index));
        }
    } else if (choice == 5) {
        running = false;
    }
}
```

Export of “StudentJobPortal.java” from VSCode MacOS taken the 13th Aug 2026.

On top of that, job information is represented in the form of **5 separate mutable static lists**, which is generally **not good practice**, because the **program is expecting very specific information**, and if that **information isn't filled out as it's expected** (*for some reason*) this could **cause errors**, or worst-case scenario, **crashes** further down the line.

This is known as “**breaking encapsulation**”, because there is **no “Job” object** responsible for **protecting and validating** the state belonging to a single job; instead, a job only exists through **matching indexes across several different lists**.

For example, in this instance:
“*jobTitles.get(i)*”, “*companies.get(i)*”,
“*jobTypes.get(i)*” etc...

If a programming error occurs that adds an element to one of the lists, it would place the system in to an “**error / inconsistent state**”.

next page >>

```
public class StudentJobPortal {
    static List<String> jobTitles = new ArrayList<>();
    static List<String> companies = new ArrayList<>();
    static List<String> jobTypes = new ArrayList<>();
    static List<String> locations = new ArrayList<>();
    static List<Boolean> savedJobs = new ArrayList<>();

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean running = true;

        jobTitles.add(e: "Java Developer");
        companies.add(e: "Tech Solutions Ltd");
        jobTypes.add(e: "Graduate");
        locations.add(e: "Cheltenham");
        savedJobs.add(e: false);

        jobTitles.add(e: "Software Tester");
        companies.add(e: "SecureApps UK");
        jobTypes.add(e: "Placement");
        locations.add(e: "Bristol");
        savedJobs.add(e: false);

        while (running) {
            System.out.println(x: "\n1. View Jobs");
            System.out.println(x: "2. Search Jobs");
            System.out.println(x: "3. Save Job");
            System.out.println(x: "4. Apply for Job");
            System.out.println(x: "5. Exit");
            System.out.print(s: "Select option: ");
            int choice = scanner.nextInt();
            scanner.nextLine();
        }
    }
}
```

*Export of “StudentJobPortal.java” from VSCode
MacOS taken the 13th Aug 2026.*

The updated version of the program **addresses these issues** by introducing an **immutable domain class**, and a job is now represented using the following;

```
8 public final class Job {
11     private final JobID id;
12     private final String title;
13     private final String company;
14     private final String jobType;
15     private final String location;
16
17     // single list for job access
18     private Job(
19         JobID id,
20         String title,
21         String company,
22         String jobType,
23         String location) {
24
25         this.id = id;
26         this.title = title;
27         this.company = company;
28         this.jobType = jobType;
29         this.location = location;
30     }
```

This on the other hand, **does demonstrate encapsulation**, because first of all; **the internal state is private** which means it **can't be modified**, it can **only be read using controlled "accessor methods"** such as **"getTitle()"** to get the job title. This also **demonstrates immutability**, because **fields are marked as "final"**, which means the classes expose no setters, and a **valid job can't be changed unexpectedly by another part of the application**. Their construction is controlled through the **"Job.Builder()"**, which **validates identifiers and text fields** before allowing a "job" to be made thanks to its **"build()"** method.

Extract from "src/main/java/com/studentjobportal/model/Job.java" on the 13th Aug 2026.

This moves me swiftly to my implementation of the **Gang of Four (G.o.F.) Builder Creational Pattern**. It's appropriate here because the object in question (*a job*) **contains** several **related properties** and **validation rules**. This prevents us from having a **multitude of indexed lists** to create a **single job** and makes **construction much clearer**.

```
32 // accessor methods
33 public static Builder builder()
34 |     return new Builder();
35 |
36 |
37 public JobID getId() {
38 |     return id;
39 | }
40 |
41 public String getTitle() {
42 |     return title;
43 | }
```

Extract from "src/main/java/com/studentjobportal/model/Job.java" on the 13th Aug 2026.

next page >>

The same approach is used in “*model/Application.java*” where “*Application*” is also **immutable** and created using the “*Application.Builder*”.

```
9 private final UUID value;
15 public static ApplicationID generate() {
16     return new ApplicationID(UUID.randomUUID());
17 }
18
19 public static ApplicationID from(String value) {
20     Objects.requireNonNull(value, "Application ID value cannot be null");
21     return new ApplicationID(UUID.fromString(value));
22 }
```

Extracts (2) from
“src/main/java/com/studentjobportal/model/ApplicationID.java” on the 13th Aug 2026.

```
31 public Application applyForJob(JobID jobID) {
32     requireExistingJob(jobID);
33
34     if (applicationRepository.findByJobId(jobID).isPresent()) {
35         throw new DuplicateApplicationException(jobID);
36     }
37
38     Application application = Application.builder()
39         .id(ApplicationID.generate())
40         .jobID(jobID)
41         .status(ApplicationStatus.SUBMITTED)
42         .submittedAt(Instant.now(clock))
43         .build();
44
45     if (!applicationRepository.save(application)) {
46         throw new DuplicateApplicationException(jobID);
47     }
48
49     return application;
50 }
```

Extract from
“src/main/java/com/studentjobportal/services/ApplicationService.java” on the 13th Aug 2026.

Value Objects are yet another **improvement in encapsulation**, more specifically the “*JobID*” and “*ApplicationID*”.

Instead of identifying each job or job application with an **indexed list number (i)**, **both encapsulate a UUID**.

Also, their **creation is controlled** through a **generator** like “*ApplicationID.generate()*” for example, which **improves safety & maintainability** due to the methods used: such as “*applyForJob(JobID jobID)*” which **explicitly require a JobID** rather than an **indexed value (i)**.

Another problem with the initial code, is that there was **no visible abstraction**, and **all storage, searching, input, output and application behaviour** more generally, **were implemented in “main()”**.

A good example of this would be the search function in the provided “*StudentJobPortal.java*” file:

```
44 } else if (choice == 2) {
45     System.out.print(s: "Enter keyword: ");
46     String keyword = scanner.nextLine().toLowerCase();
47     for (int i = 0; i < jobTitles.size(); i++) {
48         if (jobTitles.get(i).toLowerCase().contains(keyword) ||
49             companies.get(i).toLowerCase().contains(keyword) ||
50             locations.get(i).toLowerCase().contains(keyword)) {
51             System.out.println("Match: " + jobTitles.get(i));
52         }
53     }
54 } else if (choice == 3) {
```

Export of “*StudentJobPortal.java*” from VSCode MacOS taken the 13th Aug 2026.

next page >>

Any extension, modification or replacement of this search method would **require editing this conditional directly**. The refactored version of the program **replaces** this direct conditional with the “**JobSearchStrategy**” interface shown below.

```
5    // Replace-able job matching algorithm
6    // Blank = nothing.
7
8    public interface JobSearchStrategy {
9        boolean matches(Job job, String searchTerm);
10    }
```

Extract from “src/main/java/com/studentjobportal/search/JobSearchStrategy.java” on the 13th Aug 2026.

This concrete implementation allows for the inclusion of “*TitleSearchStrategy*”, “*CompanySearchStrategy*”, “*LocationSearchStrategy*” and finally “*JobTypeSearchStrategy*” which can **each have their own “algorithms”**. As an example to this; “*TitleSearchStrategy.matches()*” is relevant to the following algorithm: “*return SearchSupport.contains(Job::getTitle, job, searchTerm);*”.

```
5    public final class TitleSearchStrategy implements JobSearchStrategy {
6
7        @Override
8        public boolean matches(Job job, String searchTerm) {
9            return SearchSupport.contains(Job::getTitle, job, searchTerm);
10    }
```

Extract from “src/main/java/com/studentjobportal/search/TitleSearchStrategy.java” on the 13th Aug 2026.

This is also a good example of the inclusion of **Polymorphism** due to the fact that the “*JobService*” **works with** the “*JobSearchStrategy*” interface **rather than needing to know which search algorithm has been supplied**. This also serves as the **second Gang of Four (G.o.F.)** implementation, this time the **Strategy Behavioural Pattern**. It’s appropriate here because **searching is inherently expected to vary** based on what the **user needs**; and **alternative strategies** can be **substituted without rewriting the “JobService”**.

This is great for our requirements of improving **extendibility** because another implementation, such as “*SalarySearchStrategy*”, could implement “*JobSearchStrategy*” while the existing service **remains untouched**.

next page >>

This also allowed me to make the “*CombinedKeywordSearchStrategy*” which further demonstrates **composition over inheritance** by holding multiple “*JobSearchStrategy*” objects

```
12     private final List<JobSearchStrategy> strategies = Arrays.asList(  
13         new TitleSearchStrategy(),  
14         new CompanySearchStrategy(),  
15         new LocationSearchStrategy(),  
16         new JobTypeSearchStrategy()  
17     );
```

Extract from

“src/main/java/com/studentjobportal/search/CombinedKeywordSearchStrategy.java” on the 13th Aug 2026.

Another key **abstraction** in the revised “*StudentJobPortal*” application is the repository layer. These interfaces **hide how information is actually stored**, whereas **classes** containing “*InMemory...*” supply a **current implementation**. The following example is given:

```
10     // store a max of 1 job per application  
11     public interface ApplicationRepository {  
12         boolean save(Application application);  
13         Optional<Application> findById(JobID jobId);  
14         List<Application> findAll();  
15     }
```

Extract from “src/main/java/com/studentjobportal/repository/ApplicationRepository.java” on the 13th Aug 2026.

The above example, applies the **repository pattern**, which hides the “*LinkedHashMap*” used by the **in-memory implementation**. This allows for a **later upgrade** to a full **database-backed** system all whilst **not affecting** any of the service layer because they continue to **depend on the same interface**.

next page >>

```

1 import java.time.Clock;
2 import java.util.Scanner;
3
4 import com.studentjobportal.cli.JobPortalCli;
5 import com.studentjobportal.data.SampleJobDataSeeder;
6 import com.studentjobportal.repository.ApplicationRepository;
7 import com.studentjobportal.repository.InMemoryApplicationRepository;
8 import com.studentjobportal.repository.InMemoryJobRepository;
9 import com.studentjobportal.repository.InMemorySavedJobRepository;
10 import com.studentjobportal.repository.JobRepository;
11 import com.studentjobportal.repository.SavedJobRepository;
12 import com.studentjobportal.search.CombinedKeywordSearchStrategy;
13 import com.studentjobportal.service.ApplicationService;
14 import com.studentjobportal.service.JobService;
15 import com.studentjobportal.service.SavedJobService;
16
17 // Starting point with little to no responsibility
18 public final class StudentJobPortal {
19     private StudentJobPortal() {
20     }
21
22     @RunMain(DebugMain) Run(DebugMain)
23     public static void main(String[] args) {
24         JobRepository jobRepository = new InMemoryJobRepository();
25         SavedJobRepository savedJobRepository = new InMemorySavedJobRepository();
26         ApplicationRepository applicationRepository = new InMemoryApplicationRepository();
27
28         SampleJobDataSeeder.seed(jobRepository);
29
30         JobSearchStrategy searchStrategy = new CombinedKeywordSearchStrategy();
31         JobService jobService = new JobService(jobRepository, searchStrategy);
32         SavedJobService savedJobService = new SavedJobService(jobRepository, savedJobRepository);
33         ApplicationService applicationService = new ApplicationService(jobRepository, applicationRepository, Clock.systemUTC());
34
35         try (Scanner scanner = new Scanner(System.in)) {
36             JobPortalCli cli = new JobPortalCli(scanner, System.out, jobService, savedJobService, applicationService);
37             cli.run();
38         }
39     }
40 }

```

Extract from new “StudentJobPortal.java” on
the 13th Aug 2026

This brings me swiftly to the “*Single Responsibility Principle*”. The provided source code had all of the program in a **single file**: “*StudentJobPortal.java*”, however, the new, updated system **divides these responsibilities between several focussed classes**.

- *JobPortalCli*
- *JobService*
- *SavedJobService*
- *ApplicationService* &
- *SampleJobDataSeeder*

This means that the new **refactored “*main()*”** is **largely empty**, and only contains the **composition root**.

To summarise, the original version of the StudentJobPortal **technically** used a java class but it **behaved as a procedural, single file program with shared mutable states, parallel connections, hard coded algorithms and tightly coupled input & output**. The revised program has a much **stronger object-oriented design** with **Encapsulation** to **protect domain states** through the use of **private immutable fields**; **Abstraction** is available through **repository** and **strategy** interfaces; and **Polymorphism** that allows for **multiple repository and search implementations** to be used.

The inclusion of **immutability, value objects, defensive collections** and **composition over inheritance** further improve the **systems testability, maintainability** and **extendibility**.

The **Gang of Four (G.o.F.) Builder** and **Strategy** patterns have been implemented and the “*CombinedKeywordSearchStrategy*” showcases a **composite style structural design**. Additional **Repository** and **Service Layer** patterns separate **persistence** and **business logic** even though they aren’t part of the original Gang of Four (G.o.F.) catalogue. They’re appropriate because they **isolate areas** of the system that are **likely to vary independently**, which results in future changes such as **alternative search algorithms, database persistence** or **additional job fields**, being introduced with significantly **less modification to existing code**.

next page >>

Part 2:

The original “*ModuleCatalogue*” that was provided in the source code stored “*OptionalModule*” objects in an array list and **searched for them sequentially** as seen below:

```
public ModuleCatalogue() {
    this.modules = new ArrayList<>();
}

public ModuleCatalogue(List<OptionalModule> modules) {
    this.modules = new ArrayList<>(modules);
}

public void addModule(OptionalModule module) {
    modules.add(module);
}

// linear search O(n)
public OptionalModule searchByCode(String code) {
    for (OptionalModule module : modules) {
        if (module.getCode().equalsIgnoreCase(code)) {
            return module;
        }
    }
    return null;
}

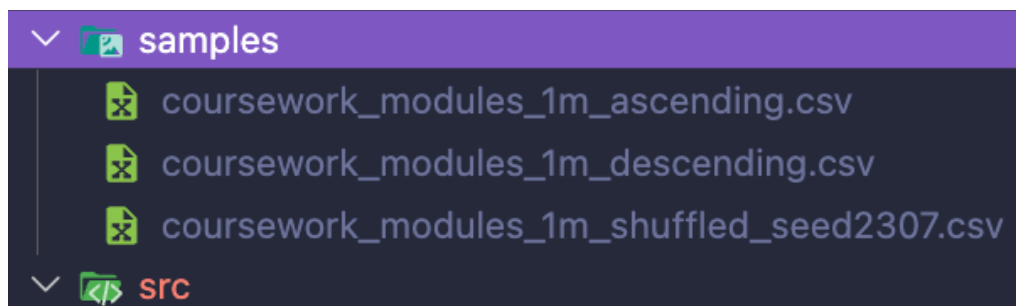
public List<OptionalModule> searchByYearAndSubject(int year, String subjectArea) {
    List<OptionalModule> results = new ArrayList<>();
    for (OptionalModule module : modules) {
        if (module.getYear() == year && module.getSubjectArea().equalsIgnoreCase(subjectArea)) {
            results.add(module);
        }
    }
    return results;
}

public List<OptionalModule> searchByKeyword(String keyword) {
    List<OptionalModule> results = new ArrayList<>();
    for (OptionalModule module : modules) {
        if (module.getCode().toLowerCase().contains(keyword.toLowerCase()) ||
            module.getName().toLowerCase().contains(keyword.toLowerCase()) ||
            module.getSubjectArea().toLowerCase().contains(keyword.toLowerCase())) {
            results.add(module);
        }
    }
}
```

Extract from new “*ModuleCatalogue.java*” on the 17th Aug 2026

This **IS the appropriate object-oriented design** for a **small data-set** due to the fact that “*OptionalModule*” **encapsulates the data for each of the modules** while “*ModuleCatalogue*” is made responsible for **managing and searching the collection**. However, on **larger data-sets** this is **not an optimal solution**, and could take **longer** than otherwise needed because it is inherently **inefficient**, especially for our tests using **1 million records**.

Speaking of which, I have created (*with the help of ChatGPT – in declaration files*) three test CSV files containing 1 million of the **same records**, but one is in **ascending order**, another in **descending order** and finally, one is in a **random order** to **compare the speed** of the created algorithms in their **best- and worst-case scenarios**.



Screenshot taken from VSCode on MacOS the 17th Aug 2026

The program that was originally given to us appeared to be a **Linear Search Algorithm**, with an **average and worst-case time complexity of $O(n)$** , which means that the existing “*searchByYearAndSubject()*” and “*searchByKeyword()*” methods also **iterated through the entirety of the dataset**.

The benchmark shows this behaviour in more detail;

For the **ascending dataset**, searching for CM0999999, located at the end of the collection, required an average of **35.93ms** using the linear search.

Similarly, searching for a module that **did not exist** required **15.57ms**, because **all 1,000,000 records had to be examined**.

In the shuffled dataset, the existing module took **2.99ms**, demonstrating that linear-search performance **depends strongly on the target's position**.

```
=====
Dataset: coursework_modules_1m_ascending.csv
=====
Records loaded: 1,000,000
CSV loading time: 502.120 ms
Binary catalogue build/sort: 34.084 ms

Existing code being searched: CM0999999
--- Existing module ---
Algorithm 1 - Linear O(n):      35,932,138 ns/search (35.932138 ms)
Algorithm 2 - Binary O(log n): 121 ns/search (0.000121 ms)
Search speed-up:              297256.68x

Missing code being searched: __MODULE_CODE_DOES_NOT_EXIST__
--- Missing module ---
Algorithm 1 - Linear O(n):      15,565,633 ns/search (15.565633 ms)
Algorithm 2 - Binary O(log n): 106 ns/search (0.000106 ms)
Search speed-up:              146525.94x

=====
Dataset: coursework_modules_1m_descending.csv
=====
Records loaded: 1,000,000
CSV loading time: 437.243 ms
Binary catalogue build/sort: 38.202 ms

Existing code being searched: CM0999999
--- Existing module ---
Algorithm 1 - Linear O(n):      4 ns/search (0.000004 ms)
Algorithm 2 - Binary O(log n): 124 ns/search (0.000124 ms)
Search speed-up:              0.03x

Missing code being searched: __MODULE_CODE_DOES_NOT_EXIST__
--- Missing module ---
Algorithm 1 - Linear O(n):      18,519,446 ns/search (18.519446 ms)
Algorithm 2 - Binary O(log n): 95 ns/search (0.000095 ms)
Search speed-up:              195030.49x

=====
Dataset: coursework_modules_1m_shuffled_seed2307.csv
=====
Records loaded: 1,000,000
CSV loading time: 386.678 ms
Binary catalogue build/sort: 927.601 ms

Existing code being searched: CM0999999
--- Existing module ---
Algorithm 1 - Linear O(n):      2,994,842 ns/search (2.994842 ms)
Algorithm 2 - Binary O(log n): 123 ns/search (0.000123 ms)
Search speed-up:              24414.72x

Missing code being searched: __MODULE_CODE_DOES_NOT_EXIST__
--- Missing module ---
Algorithm 1 - Linear O(n):      19,111,183 ns/search (19.111183 ms)
Algorithm 2 - Binary O(log n): 94 ns/search (0.000094 ms)
Search speed-up:              202865.34x
```

Screenshot taken from the CLI on MacOS the
17th Aug 2026

Now, in the updated redesign of the search algorithm, I automatically thought to implement a **Binary Search**; because I was thinking back to an old module that we were taught by *Dr. Allen Stuart*, where I was made to go in front of everyone and explain it with a visual analogy. This would introduce “*BinarySearchModuleCatalogue*”, which would create its own sorted “*ArrayList<OptionalModule>*” ordered by module code allowing for **Binary Search** to repeatedly “**eliminate**” half of the remaining searches from the list. That would make its **average and worst-case lookup complexity $O(\log n)$** . That would be **$\log_2(1000000)$ iterations** (which is **approximately 20 iterations** for **1M rows of data**, unlike the **potentially 1M iterations** that **Linear Search** would require).

next page >>

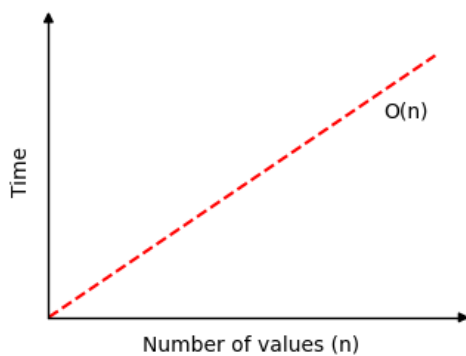


Image downloaded from W3Schools on the 17th Aug 2026

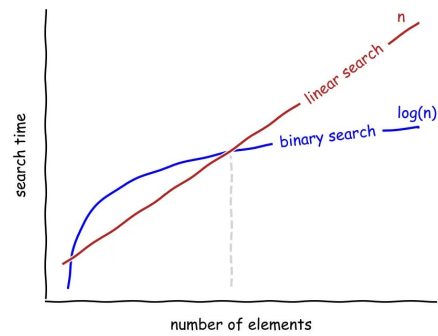


Image downloaded from "medium.com" user "Mehmet Akif Özgür" on the 17th Aug 2026

HOWEVER, binary search **REQUIRES** the list to be sorted before processing it, because it is after-all, essentially **discarding half of the given list at any time**. The sort algorithm in Java (.sort) appears to be $O(n \log n)$ according to *Baeldung* online.

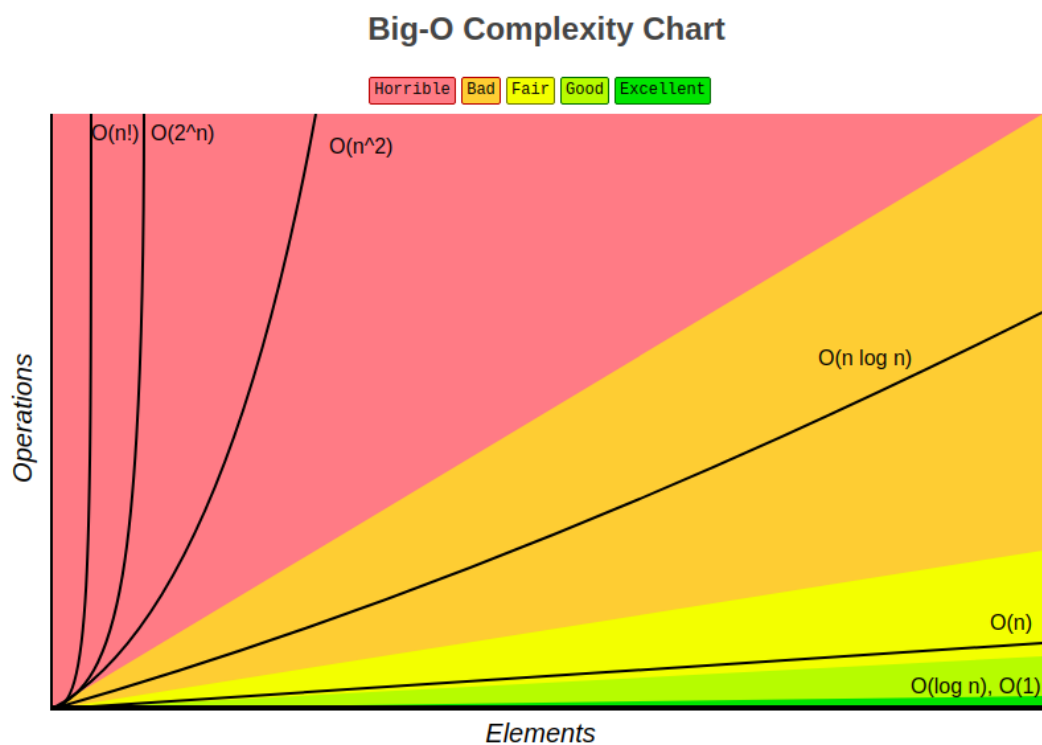


Image downloaded from *Analytics Vidhya* online on the 17th Aug 2026

But, upon testing, the new and improved benchmark showed that the Binary Search **consistently took approximately 94 to 124ns REGARDLESS** of whether or not the original dataset **was ascending, descending or shuffled**. For instance, the missing-module search on the shuffled dataset showcased that we went from **19.4ms** on the **Linear Search Algorithm** to a **whopping 94ns** using the **Binary Search**. That's around **202,865x faster!**

next page >>

```

MODULE CATALOGUE SEARCH BENCHMARK
Algorithm 1: Linear Search - O(n)
Algorithm 2: Binary Search - O(log n)

=====
Dataset: coursework_modules_1m_ascending.csv
=====
Records loaded: 1,000,000
CSV loading time: 502.120 ms
Binary catalogue build/sort: 34.084 ms

Existing code being searched: CM0999999
--- Existing module ---
Algorithm 1 - Linear O(n):      35,932,138 ns/search (35.932138 ms)
Algorithm 2 - Binary O(log n): 121 ns/search (0.000121 ms)
Search speed-up:                297256.68x

Missing code being searched: __MODULE_CODE_DOES_NOT_EXIST__
--- Missing module ---
Algorithm 1 - Linear O(n):      15,565,633 ns/search (15.565633 ms)
Algorithm 2 - Binary O(log n): 106 ns/search (0.000106 ms)
Search speed-up:                146525.94x

=====
Dataset: coursework_modules_1m_descending.csv
=====
Records loaded: 1,000,000
CSV loading time: 437.243 ms
Binary catalogue build/sort: 38.202 ms

Existing code being searched: CM0999999
--- Existing module ---
Algorithm 1 - Linear O(n):       4 ns/search (0.000004 ms)
Algorithm 2 - Binary O(log n): 124 ns/search (0.000124 ms)
Search speed-up:                 0.03x

Missing code being searched: __MODULE_CODE_DOES_NOT_EXIST__
--- Missing module ---
Algorithm 1 - Linear O(n):      18,519,446 ns/search (18.519446 ms)
Algorithm 2 - Binary O(log n):  95 ns/search (0.000095 ms)
Search speed-up:                195030.49x

=====
Dataset: coursework_modules_1m_shuffled_seed2307.csv
=====
Records loaded: 1,000,000
CSV loading time: 386.678 ms
Binary catalogue build/sort: 927.601 ms

Existing code being searched: CM0999999
--- Existing module ---
Algorithm 1 - Linear O(n):      2,994,842 ns/search (2.994842 ms)
Algorithm 2 - Binary O(log n): 123 ns/search (0.000123 ms)
Search speed-up:                24414.72x

Missing code being searched: __MODULE_CODE_DOES_NOT_EXIST__
--- Missing module ---
Algorithm 1 - Linear O(n):      19,111,183 ns/search (19.111183 ms)
Algorithm 2 - Binary O(log n):  94 ns/search (0.000094 ms)
Search speed-up:                202865.34x

```

Screenshot taken from the CLI on MacOS the 17th Aug 2026

An important **exception** occurred with the descending dataset; **CM0999999 happens to be the first element**, allowing **Linear Search** to complete in **only 4ns**. This represents its best case **O(1)** and explains why **Binary Search** was **slower** for that **individual lookup**. However, the missing-module search on the same dataset still required **18.52ms**, confirming that the **worst-case complexity remains O(n)**.

So, **about that pre-processing trade-off: Binary search requires sorted data**, so constructing the binary catalogue **costs O(n log n)**. The shuffled dataset required **927.60ms to sort**. This cost **IS worthwhile** when **many searches are performed** because the **sorted catalogue is constructed once**. While **subsequent searches remain O(log n)**.

next page >>

In conclusion, the refactored **$O(\log n)$** binary search provides a **substantial improvement** in search efficiency for **large datasets**. Although it introduces an initial **$O(n \log n)$ pre-processing cost** to **sort the data**, this is **justified** when **repeated searches are performed**. The benchmark results demonstrate this benefit, with binary search achieving **speed improvements exceeding 200,000x** over the **$O(n)$** linear search **in worst-case searches**.